This handout includes space for every question that requires a written response. Please feel free to use it to hand write your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

1.a [5 points (Coding)] Implement Fine Tune

As a warmup, we will perform perhaps the most straightforward form of k-shot learning with a pre-trained model: directly fine-tuning the entire model on the k examples. We will use two different sizes of smaller BERT models that have been compressed through distillation.

Implement the logic for fine-tuning, including selecting the parameters that will be fine-tuned (only the case for 'all' for this question) in ft.py:parameters to fine tune(), and computing the loss and accuracy in ft.py:get loss and ft.py:get acc. You only need to complete the loss/accuracy calculations under if logits.dim() == 2: for this question.

See ft.py:parameters_to_fine_tune(), ft.py:get_loss and ft.py:get_acc

Hints I used: parameters_to_fine_tune(): For the complete fine-tuning case, we are simply returning all of the learned parameters, as in:

```
return model.parameters()
```

get_loss(): Similarly:

```
F.cross_entropy(input=logits, target=targets)
```

get_acc(): Here we had to be a bit more thoughtful:

Method: match maximum value across classes (argmax), convert Boolean to float and take the mean.

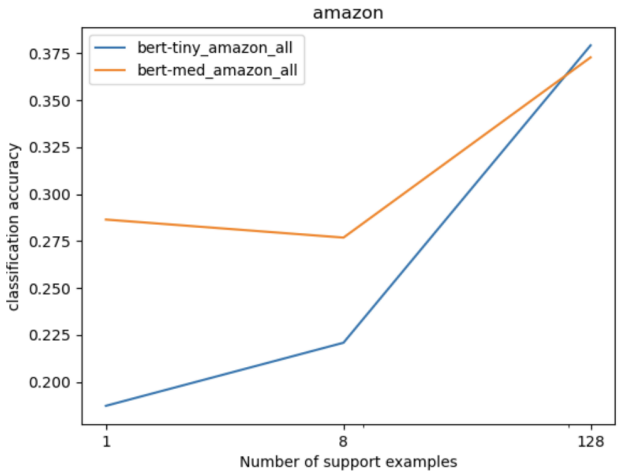
```
The usual one-liner looks like:
    acc = (torch.argmax(logits, dim=-1) == targets).float().mean().item()

Instead of this usual simple implementations, I added an additional (generally
unnecessary) softmax on the original logits (unnormalized log probabilities)
since the Slack conversation indicated we might have some environment
sensitivity / round off error propogation.
```

1.b Observe Fine-Tuning

(i) [5 points (Coding)] Run Fine-Tuning

Your plot should look as follows:



My plot via the following command is shown below.

```
python main.py --task plot_ft --model bert-tiny,bert-med --dataset amazon --k 1,8,128
```

Per this course Slack conversation on Feb-29, why the BERT medium K=1 case (orange) is different from the expected Handout is unknown and "ok" per the course facilitators.

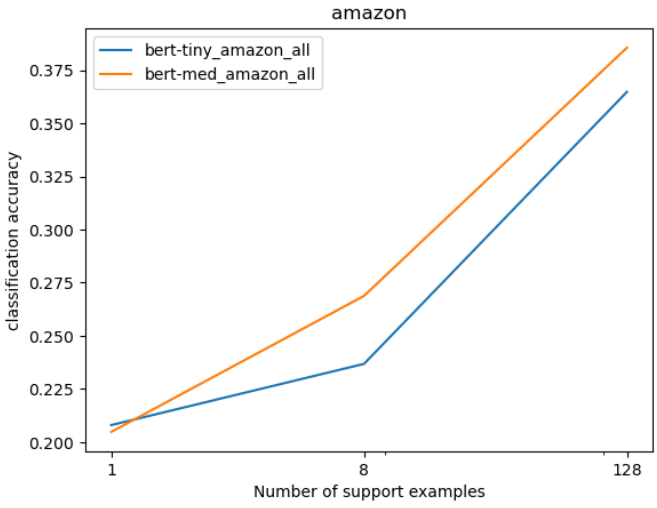


Figure 1: Performance of two sizes of BERT models on the Amazon Reviews dataset for various values of k

(ii) [2 points (Written)] Reason on Fine-Tuning

In one sentence, what do you notice about the performance of the two model scales?

Answer: Using my figure for interpretation, the performance (accuracy) is similar for both scales and roughly co-linear for the K=8 and K=128 Support data cases.

1.c [2 points (Written)] Storage for Fine-Tune

If we fine-tune all of our model parameters for each task, we must save a new complete copy of the model's parameters for each new task. As an example, a BERT-mini model similar to the ones you just fine-tuned has approximately 11.2 million parameters; assuming parameters are represented as 4-byte floats, after fine-tuning on a new task, how much disk space do we need to store the new fine-tuned model parameters?

Answer:
44.8 million bytes/task or 42.7 MBytes/task

1.d [2 points (Written)] Disk Space

Google's recent large language model PaLM has 540 billion parameters. How much disk space would be needed to store a new fine-tuned version of this model, assuming parameters are represented as 4-byte floats?

Answer:
$\approx$ 2 TB/task (4 * 540e9 / 1024**4)

2.a [8 points (Coding)] Implement In-Context Learning

A surprising property of large language models is their emergent ability to learn in-context, that is, their ability to learn a task without updating any parameters at all. The name 'in-context' comes from the fact that the learning is done by simply including several examples or a task description (or both!) prepended to a test input present to the model.

Complete the code for in-context learning in icl.py.
(i) Implement prompt creation for the XSum and bAbI tasks, which is found in icl.py:get_icl_prompts().

Key insights employed:
Nothing special, just followed all the requirements in the Handout.

(ii) Implement greedy sampling in icl.py:do_ sample().

Key insights employed:
a. When calling the model here, be sure to wrap the call with torch.inference_mode() to save memory!

b. Don't use model.generate; implement this yourself in a loop

c. Note 3: Be sure you understand what each dimension of the model's output logits represents.
Misinterpreting the dimensions of this output can lead to subtle bugs.

d. 'Greedy' sampling means that given a context $x_<t$ producing a distribution over next tokens $p\theta(x_t|x_<t)$,
we deterministically choose the next token x_t to be the token with highest probability.

e. Stop at max tokens or a stop token.

(iii) Finally, put the pieces together by completing the implementation of icl.py:run_icl(), using your get_icl_prompts() and do _sample() functions, as well as the HuggingFace tokenizer defined in the loop.

Primary steps taken: Only 4 lines of code.

a. Get prompt (ASCII text)
b. Encode prompt to tokens (integer ids) on proper device
c. Sample model
d. Decode model output tokens back to text

2.b Evaluate on bAbI

(i) [5 points (Coding)] Run ICL on bAbI

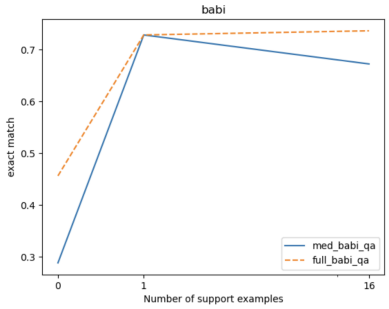
First, evaluate k-shot in-context performance on bAbI for GPT-2-medium (355M parameters) and fullsize GPT-2 (1.5B parameters) for various values of k with the command:

```
python3 main.py --task run icl --model med,full --dataset babi --k 0,1,16
```

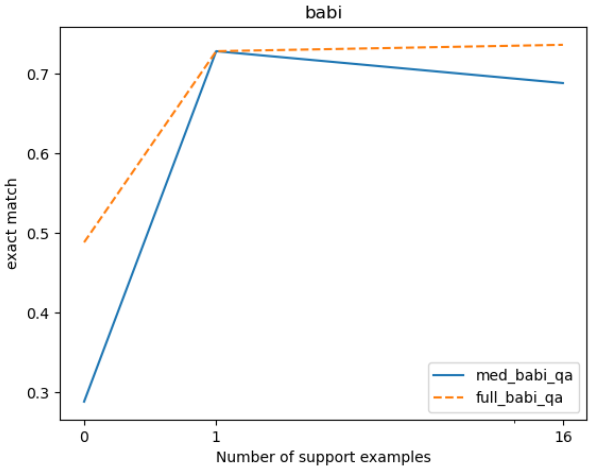
Plot the results with the command:

```
python3 main.py --task plot icl --model med,full --dataset babi --k 0,1,16
```

Your plots should look like:



My performance figure indeed looks as it should:



(ii) [2 points (Written)] Reason on ICL on bAbI

What relationship(s) do you notice between model scale and few-shot performance?

Answer:

Observations:

(a) Performance is NOT (statistically) dependent on model scale for k=1 shot.

(b) Performance improves ↑ as model scale increases ↑ in the 0-shot (k=0) and many-shot (k=16) cases.

(c) The larger model is more general than the smaller model as evidenced by comparing the 0-shot (k=0) performance points.

Intuition:

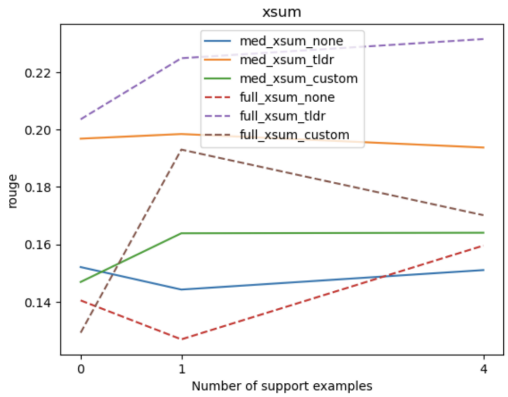
(a) In context learning is equally effective for 1-shot (k=1).

(b) The medium sized model (blue trace) lacks sufficient expressiveness to take advantage of the many-shot (k=16) in context learning prompts.

2.c Evaluate on XSum

(i) [5 points (Coding)] Run ICL on XSum
Now let's evaluate several different prompt formats on the XSum dataset. With and without a task description in the prompt, evaluate zero-shot and few-shot performance for XSum on GPT-2-Medium.

Your plot should look like:



My performance figure indeed looks as it should for the "none" (blue solid, and red dashed) and "TLDR" (orange solid and purple dashed) prompt formats:

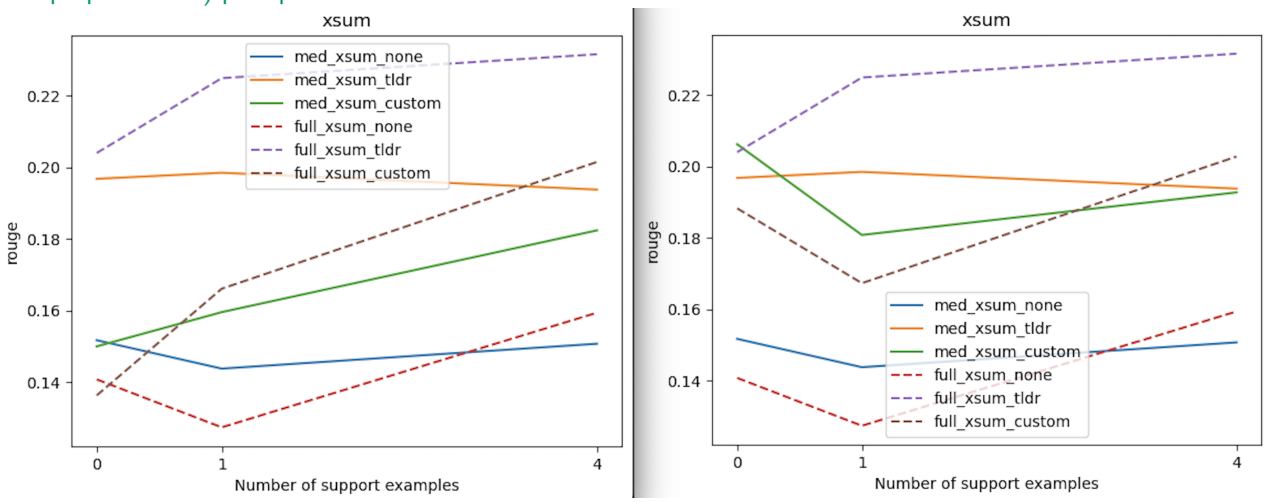


Figure 2c: Plot of *rouge* scores versus number of support examples (k=0, 1, 4 shot) for the *medium* (sold colors) and *full* (dashed lines) XSum models, for the prompts: none, "TL;DR:", and custom. On the left figure, the custom prompt is ":" and on the right figure, it is "Q:<question>A:TL;DR:In the<answer>"

2.c Evaluate on XSum Continued

(ii) [2 points (Written)] Reason on ICL on bAbI

(a) How does the performance of the TL;DR: prompt compare with no prompt formatting?
(b) What was your custom prompt format, and how did it compare with TL;DR:?
(c) Discuss the relative performance of the different prompts in the zero-shot, one-shot, and few-shot settings.

Answer:
(a) The "TL;DR: " prompt performs much better than the no-prompt formatting in all k-shot cases for both model sizes (solid: orange vs. blue, and dashed: purple vs red).

(b) For my custom prompt, I initially chose an extremely simple prompt, specifically " :" just to benchmark the comparison (aforementined Figure 2c left). It compares worse than "TL;DR:", as expected due to it's simplicity.
However, looking at the few-shot (k=4) case for the medium and full model (orange versus green, and purple versus brown), there is an indication that for greater shots, perhaps 8, my simple " :" custom prompt might be just as performant as "TL;DR:"–of course, the goal is to be performant at few and 0 shot learning. The bottom line is that something simple but bettern chosen than merely " :" might be able to overtake "TL;DR:" at k=4 and perhaps smaller values of k.

E.g. "Sandra travelled to the kitchen. John went to the office. Where is Sandra? TL;DR: kitchen. Sandra went to the office. Daniel went back to the garden. Where is Sandra? TL;DR:"

versus:
E.g. "Sandra travelled to the kitchen. John went to the office. Where is Sandra? : kitchen. Sandra went to the office. Daniel went back to the garden. Where is Sandra? :"

Then I tried a more complex prompt format, specifically prepending "Q:" to the question, and "A:TL;DR:In the" to the answer, as in:
"Q:<question>A:TL;DR:In the<answer>"
The performance comparisons are shown in the right-most figure in Figure 2c, previous page. Now this new custom prompt (green for medium, brown for full) is much more performant than the simpler " :" custom prompt. It is slightly better than "TL;DR:" for the 0-shot case (green vs. orange), and it is equivalent in performance to "TL;DR:" for the 4-shot case (green vs. orange) for the medium model. Otherwise this new prompt is still worse in performance than the bechmark "TL;DR:".

(c)
"few-shot": The rank ordering of best to worst is: "TL;DR:" > Custom > None. And projections to k>4-shot are roughly co-linear in the full model. As indicated above, the simple " :" prompt performance slope at k=4 is close to overtaking the performance of "TL;DR:" in the medium models–notably a less useful outcome than high performance at 0- and 1-shot.
"1-shot": The comparison is similar for 1-shot as for the few-shot case; nothing new to add.
"0-shot": This is where things are interesting, or perhaps the comparison was predictable. The overly simple custom prompt " :" is not any more useful than no prompt at all (the "none" case). So to beat "TL;DR:" obviously (and as predicted), we'd have to do a lot better at prompt engineering. This "custom" prompt is effectively in the null case–as predicted.

3 Parameter-efficient fine-tuning

3.a [5 points (Coding)] Implement Parameter-efficient Fine Tine

Finish the implementation for each version of parameter-efficient fine-tuning for GPT-2-Medium in ft.py:parameters_to_fine_tune()
(i) last: Fine-tune only the last 2 transformer blocks
(ii) first: Fine-tune only first 2 transformer blocks
(iii) middle: Fine-tune only middle 2 transformer blocks

Insights:
The key insights here are to figure out how to find the transformer blocks in them model structure, then to return the result as an iterable.

The transformer blocks are simply under model.transformer. And there are a total of 24 transformer blocks, so be careful with indexing in the "middle" case.

Then:
last:

```
[p for p in model.transformer.h[-2:].parameters()]
```

first:

```
[p for p in model.transformer.h[:2].parameters()]
```

middle:

```
[p for p in model.transformer.h[11:13].parameters()]
```

3.b [2 points (Written)] LoRA

(i) For a single layer, what are the parameter savings we achieve by using LoRA? i.e., what is the ratio of parameters fine-tuned by LoRA (for arbitrary p) to the number of parameters in W_l^0?

(ii) In terms of p, d_1, d_2, when will LoRA provide the greatest savings in newly-created parameters?

Answer:
(i) Since the rank of AB^T is p, and the rank of W_l^0 is $d1 * d2$, the the ratio is: $p^2/d_1 * d_2$

(ii) LoRA will provide the greatest savings when $p << d_1, d_2$.

3.c [14 points (Coding)] Implement LoRA

Answer: See ft.py.

A few key insights employed:

In ft.py:LoRAConv1DWrapper(...):

From Figure 1 of the LoRA paper:
"We use a random Gaussian initialization for A and zero for B, so W = BA is zero at the beginning of training"
https://arxiv.org/pdf/2106.09685.pdf

Thus:

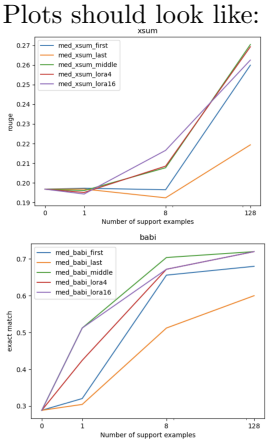
1. Freeze pre-trained weights: W_l^0
self.base_module.requires_grad_(False)

2. Create LoRA A matrix: Use Kaiming initialization rather than Gaussian distribution:
self.lora_A = nn.Parameter(torch.zeros(d1,lora_rank))
nn.init.kaiming_uniform_(self.lora_A)

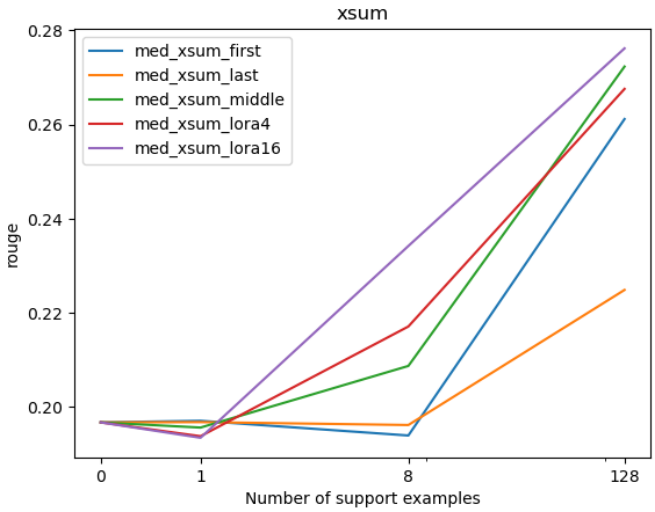
3. Create LoRA B matrix: Initizalize with simple 0's
self.lora_B = nn.Parameter(torch.zeros(d2,lora_rank))

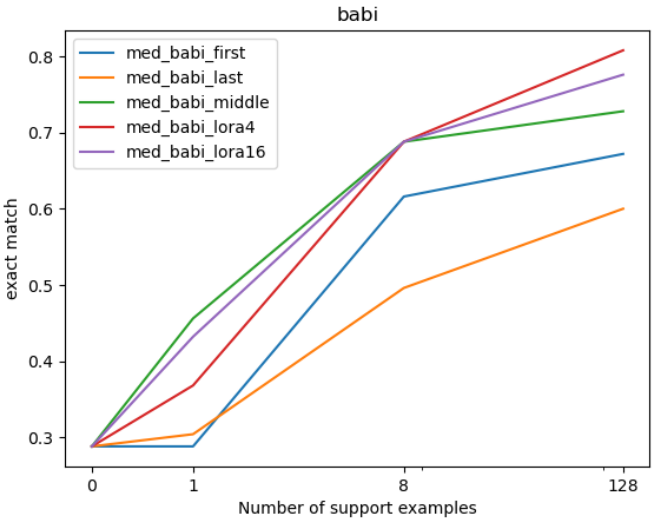
3.d Eval: (i) [5 points (Coding)] Run Fine-Tuning w/LoRA

Run fine-tuning for each parameter-efficient fine-tuning method, using p = 4, 16 for LoRA (so, 5 variants in total). Plot k-shot performance as k is varied for GPT-2-medium, one plot for each dataset.
Plots should look like:



My performance plots looks close to the benchmark–the general trends are agreeable:





3.d Evaluate: (ii) [2 points (Written)] Reason on Fine-Tuning with LoRA

Describe the results here.

Answer: For these tasks and datasets, in general, it is better to either use LoRA or finetune on the earlier layers in the model than the final layer (orange curve). This is true for both the XSum dataset (news articles from the BBC) and bAbI dataset (for reasoning about contextual information). This is slightly surprising for me as I've been reading and hearing from sources in the community that finetuning on the outer layers is always better.

Additionally, the LoRA4 (rank p=16) and LoRA16 (rank p=16) methods are very performant, either beating (k=8 on XSum) or nearly matching the best layer finetuning performances.

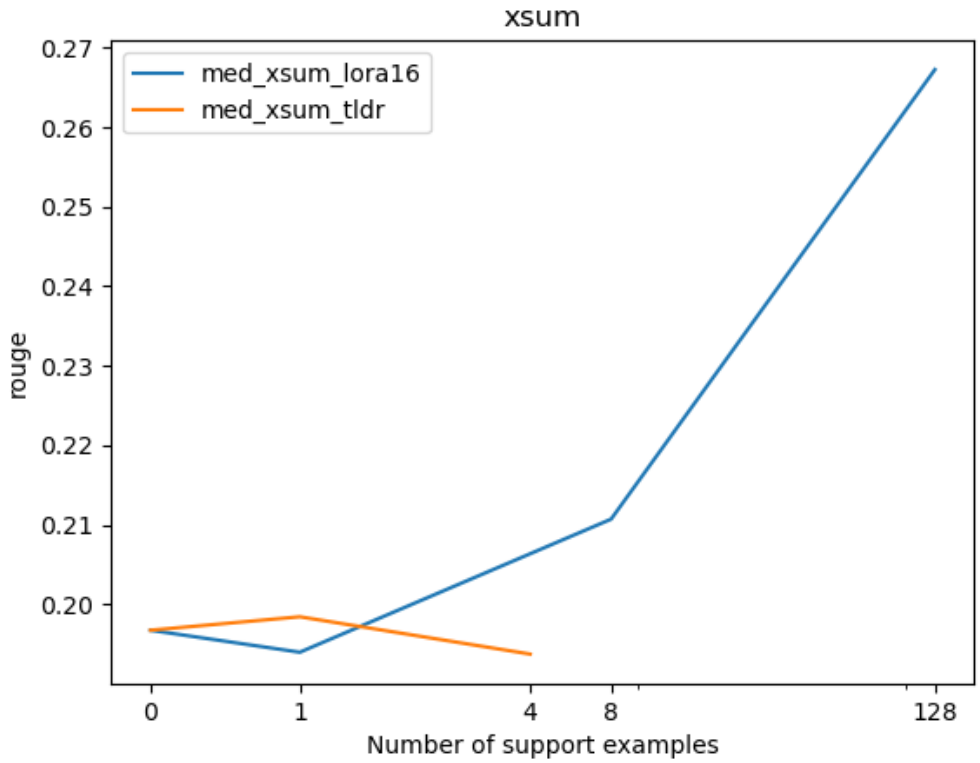
4 Comparing in-context learning and fine-tuning

4.a [2 points (Written)] Compare LoRA and In-Context Learning for XSum

(i) When does in-context learning seem like the better choice, with respect to the amount of data available?

(ii) What about fine-tuning? What limitation of in-context learning does this result highlight?

Answer:
My performance figure comparing finetuning (LoRA16 on XSum) and in-context learning (TLDR prompt on XSum):



(i) In-context learning works best when we have 1-shot, i.e. one support example (see orange versus blue at k=1). It also works "ok" when we a few support examples (i.e. few-shot) and don't have the ability to finetune–when we lack high performance computing (compute, storage, time) or other reasons (e.g. model isn't open).

(ii) If we have few to many support examples, and high performance computing available, finetuning is the better strategy by far and increasingly so when the number of support examples (k) increases.

4.b [2 points (Written)] Ordering

One potential disadvantage of in-context learning is that we must choose an ordering for the examples in our prompt, and that ordering can sometimes impact performance negatively.

Run the command to compute the evaluation performance of in-context few-shot performance for 5 random orderings of the prompt. Report the number you get; the standard deviation of performance for **fine-tuning is approximately 0.013**.

```
python3 main.py --task run_icl --model med --dataset babi --k 16 --repeats 5
```

Does in-context learning or fine-tuning have a higher standard deviation?

Answer: In-context learning has a far greater standard deviation, proving the point: finetuning is far superior. Even with 5 repeats, the best in-context standard deviation is >>than finetuning (0.013).

Evaluation performance of in-context for 5 repeats:
The numbers shown are the standard deviation.

```
Running in-context learning with med on babi with k=16 and prompt_mode=qa
Evaluation results: {'med_babi_16_qa': 0.688}
Beginning repeat #1
Evaluation results: {'med_babi_16_qa': 0.728}
Beginning repeat #2
Evaluation results: {'med_babi_16_qa': 0.712}
Beginning repeat #3
Evaluation results: {'med_babi_16_qa': 0.696}
Beginning repeat #4
Evaluation results: {'med_babi_16_qa': 0.672}
```

4.c [5 points (Written, Extra Credit)] Extra Credit

See this Lu et al., 2022 paper for multiple heuristics for picking a prompt ordering. Implement either the global entropy (GlobalE) or local entropy (localE) heuristic, and report the accuracy you find for that ordering for 16-shot bAbI on GPT-2-medium. Compare it with the accuracy you found with random prompt orderings in question 1.

Answer:

Additional Details: